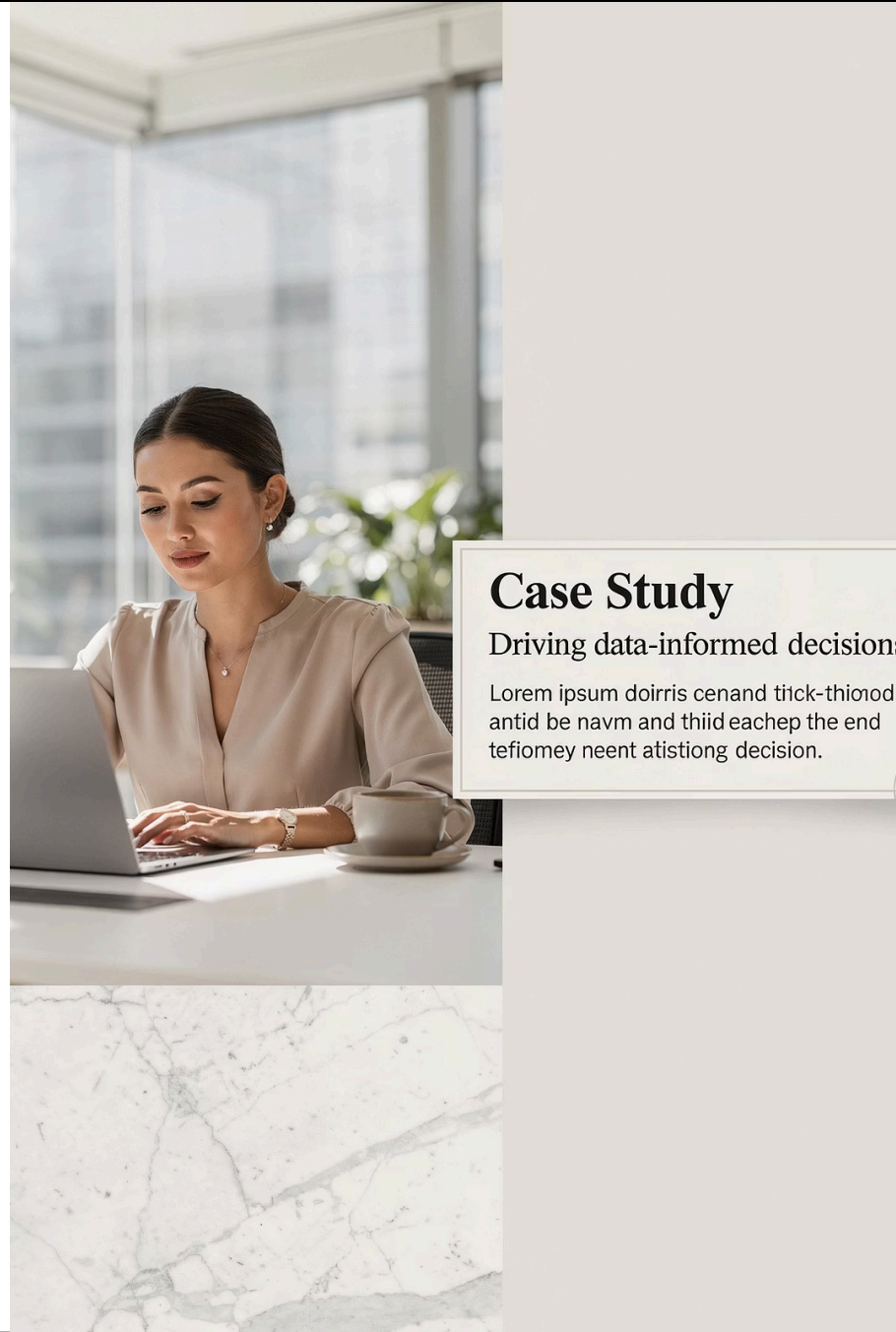
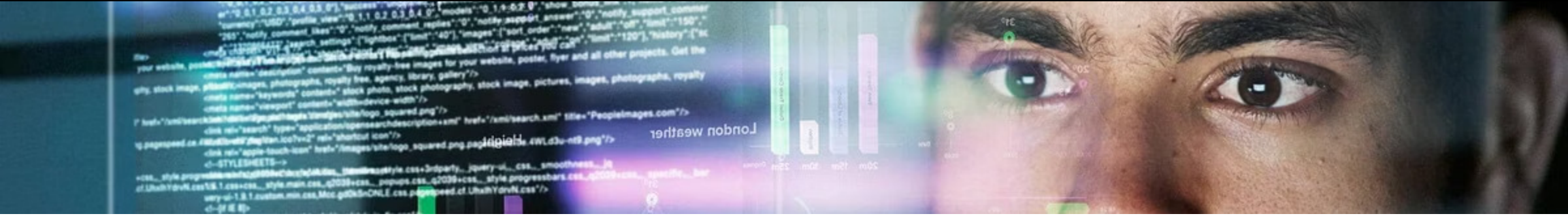
CASE STUDY

# SaaS adoption plateaus when users have to "drive the UI" instead of reaching an outcome.

A voice command and control layer built for a SaaS product where users analyze behavior data and act on what they find — without losing time to repetitive clicks.





# What this was about

This project was built for a SaaS product where users do two things every day: they analyze behavior data (heatmaps and recordings) to find friction, and then they move into setup screens (segments, page groups, variations) to act on what they found.

The product already had AI help, but teams were still losing time because the hardest part was not "knowing what to do," it was translating intent into 15 separate clicks across filters, segments, page rules, and setup screens, and then repeating the same sequence again the next day for a new page or a new audience.

We shipped a voice command and control layer that lets users speak the steps the way they would tell a teammate, but it does not behave like a magic button that silently changes things. It behaves like a senior operator: it prepares the change plan in the product UI, shows exactly what it is about to do, and only applies anything after the user confirms.

## Build posture line

If the system cannot show a clear "this is what I'm going to change" preview in the UI, it does not execute; it asks one clarifying question, or it stops and leaves the action as a draft.

# The scenes we built around

These are not generic "analysis" stories. These are the daily moments where teams waste hours and then argue about what was actually configured.

## Scene 1 — A behaviour analyst is watching 30 recordings and cannot keep context

A CRO or product analyst opens session recordings because a checkout drop has spiked, and they start with the same manual ritual: pick a date range, pick a device, pick a traffic source, filter out returning users, then try three different segments until the recordings stop looking random and start showing a repeating failure pattern.

In the middle of this, they find a clip where a user rage-clicks the promo code field, backs out, scrolls, and abandons, and the analyst wants to do two things immediately while the context is fresh: save that moment as an internal artifact and pull the matching heatmap view for the exact same audience, without going back to rebuild the segment again.

This is where work usually breaks. People either forget what segment they used, or they rebuild a slightly different one, or they take screenshots that nobody can reproduce later, and what started as "we saw a pattern" turns into "we think we saw a pattern."

**What the voice layer changed:** The analyst can say, while watching the recording, "Filter to mobile users from paid search in the last seven days who reached checkout but did not complete payment, then show me the first ten clips with repeated clicks on promo code." The system fills the filters and segment conditions in the product UI, shows the exact segment definition it created, and asks for confirmation before it applies it — because that is the difference between "I think we filtered it right" and "this is the saved segment everyone can reuse." Then when the analyst says, "Bookmark this clip and label it promo code confusion," the system creates a bookmark artifact in the product that the team can click later.

# Scene 2

These are not generic "analysis" stories. These are the daily moments where teams waste hours and then argue about what was actually configured.

## Heatmaps show friction, but teams lose time moving from insight to action

After the recordings, the analyst opens heatmaps for the same pages because they want to confirm whether the friction is real at scale, not just one clip, and the heatmap view becomes another round of manual work: apply the same segment again, open the click map, then open rage clicks and dead clicks, then cross-check scroll depth, then write a summary for the team in a doc or a ticket.

The problem is not that the work is hard. The problem is that it is repetitive and fragile. If the audience definition changes slightly, the heatmap story changes, and now the team argues about which view was used.

**What the voice layer changed:** The analyst says, "Open click heatmap for the same audience on checkout pages, then highlight dead clicks and rage clicks for the promo code area." The system takes the saved segment from the recordings view, applies it to the heatmap view, and shows a "segment applied" proof card so the analyst can later point to the exact lens used. Then the analyst says, "Create a summary note for the team with three bullets and attach the bookmarked recording." The system drafts the note inside the tool as an inspectable artifact tied to the segment and the page.

# Scene 3

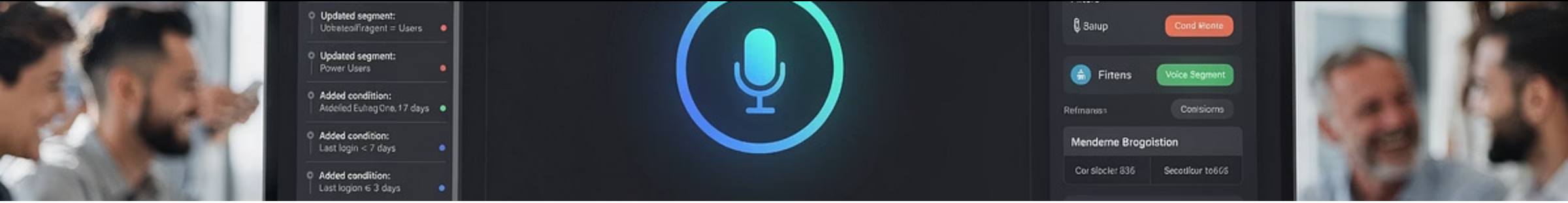
These are not generic "analysis" stories. These are the daily moments where teams waste hours and then argue about what was actually configured.

## Campaign setup becomes slow because page groups and variations are configuration-heavy

Once the team agrees that the promo code field is creating confusion, they need to act, and acting is where setup friction lives. The user must define page groups for checkout and cart URLs, exclude thank-you pages, ensure the rule pattern does not accidentally include unrelated paths, then set up a variation that moves the promo code entry or adds helper text, then name it correctly, then ensure it is tied back to the insight that triggered it.

This is where tiny mistakes waste days. A page group that accidentally includes a different flow, or a segment that is slightly off, or a variation description that is vague, and now the campaign is misconfigured and the team finds out late.

**What the voice layer changed:** The user says, "Create a page group for cart and checkout pages, exclude thank-you, and show me the rule you are about to save." The system generates the rule pattern and shows it before saving, because pattern mistakes are the silent killer of setup accuracy. Then the user says, "Draft a test variation that moves promo code below the order summary and adds a one-line hint that it can be applied at payment step." The system fills the variation brief and the setup fields, shows the resulting plan as a structured draft, and waits for a human click to confirm and proceed.




# What we actually delivered

This was not a standalone app and it was not a new UI. It was a voice layer embedded inside the product surfaces users already use, designed to turn spoken intent into visible, reviewable actions.

## Where it sits

- In the recordings and heatmap workspaces as a mic input plus a command history drawer, so a user can speak filters, segments, and analysis actions while staying inside the analysis context.
- In the campaign setup workspace as a mic input that fills page group rules and variation setup drafts, so the user can move from insight to build without losing continuity.

## What it reads

- The current analysis context like page, date range, device split, traffic source, and saved segments.
- Existing configuration context like page groups and variation templates.

## What it writes back

- Saved segments and applied filters as reusable assets, not temporary view states.
- Bookmarks and labels for recordings so teams can replay and align.
- Draft page groups and draft variation setup plans that require human confirmation before they are saved or launched.
- A visible activity log inside the tool that shows what the user said and what the system prepared, so teams can trust the trail.

# The safety rule that made teams adopt it

> The system did not try to be clever by silently acting. It behaved in a way teams already trust: it prepared the work, made the work visible, and waited for a deliberate confirmation click before any irreversible change was applied.

## What we learned

### Learning 1 — Voice speed + visible proof

The biggest adoption accelerator was not "voice speed." It was the combination of voice speed plus visible proof, because users are happy to speak commands if they can immediately see what the system understood and they can fix it before it becomes a bad configuration.

### Learning 2 — Analysis artifacts matter

The second learning was that "analysis artifacts" matter as much as "actions," because teams do not only need results, they need evidence they can forward internally without re-explaining what lens was used.

# Talk to us.

connect via neil.voishift@klyrr.com

- Do your users know what they want to do inside your product, but still need too many clicks across filters, nested settings, and configuration screens to get it done?
- Do your users create "insights" in one place, then lose momentum when they have to jump into setup screens and rebuild the same context again?
- Do your power users end up teaching new users by screen recording instead of your product onboarding them smoothly in-product?
- Do you want a voice layer that does more than dictation, meaning it can take spoken intent, fill the right screens, keep everything reviewable, and use usage data to recommend the next best action inside the product?

We will pick one workflow your users repeat weekly, embed a voice command layer directly into the screens they already use, and ship it with confirmation-first controls so adoption goes up without creating risk.